

B 2.4.5 Recursive Algorithms (HL Only)



B 2.4.5 Construct and trace recursive algorithms in a programming language. (HL only)

Become a recursion master! This section dives deeper into recursive programming, focusing on how to construct and trace simple, non-branching recursive algorithms in Java and Python. Sharpen your problem-solving skills and learn to implement elegant recursive solutions.

Learning Objectives

- Students will be able to identify the base case and recursive case in a recursive algorithm.
- Students will be able to construct and trace simple, non-branching recursive algorithms in Java and Python.
- Students will be able to implement non-branching recursive algorithms in programming

IB ATL Skills:

Thinking: Critical thinking, creative thinking

Communication: Communication, collaboration

Self-Management: Organisation, time management

Lesson Plan (2 hours):

Part 1: Non-Branching Recursion (30 minutes)

Engage (10 minutes)

Recap: Briefly review the fundamentals of recursion, including the base case and recursive case.

Non-branching vs. branching: Explain the difference between non-branching recursion (where a function makes only one recursive call per execution) and branching recursion (where a function makes multiple recursive calls).

Examples: Provide simple examples of non-branching recursive functions, such as calculating the sum of a list or finding the greatest common divisor (GCD) of two numbers.

Explore (20 minutes)

Problem-solving: Present a variety of problems suitable for non-branching recursion.

Group work: Divide students into groups and assign each group a different problem to solve using non-branching recursion.

Part 2: Code Implementation and Tracing (45 minutes)

Explain (20 minutes)

Code implementation: Have each group present their recursive solutions in either Java or Python.

Walkthrough: Guide the students through the code, explaining how the base case and recursive case work together to solve the problem.

Elaborate (25 minutes)

Tracing: Demonstrate how to trace the execution of the recursive functions with different inputs. Use visual aids like diagrams or tables to help students visualize the recursive calls and the changing values of variables.

Part 3: Practice and Q&A (45 minutes)

Evaluate (20 minutes)

Individual practice: Provide additional practice problems for students to solve individually using non-branching recursion.

Challenge: Encourage students to try optimizing their recursive solutions or explore alternative recursive approaches.

Conclusion (25 minutes)

Q&A: Facilitate a class discussion to address any remaining questions students have about non-branching recursion.

Key takeaways: Reinforce the importance of identifying the base case and recursive case, and emphasize the techniques for tracing and understanding recursive functions.

Lesson Resources

Presentations

Java

[Link to presentation](#)

Python

[Link to presentation](#)

eBook

The eBook pages can help to introduce the topic to students, present during a lesson or for students to review the content from the lesson.

B 2.4.5 Java

B 2.4.5 Python

Worksheets

Worksheets can be given at the end of the session to develop and assess student understanding.

Worksheet 1 has exam style questions that get students to trace iterative methods and turn those methods into recursive solutions.

The second worksheet is more challenging as gets students to create recursive methods from problem statements and then trace and explain the methods. Some students may need support to solve the algorithms.

B 2.4.5 Worksheet 1 Java

B 2.4.5 Worksheet 1 Python

B 2.4.5 Worksheet 1 answers

B 2.4.5 Worksheet Java

B 2.4.5 Worksheet Python

B 2.4.5 Worksheet Answer Key

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**